CO2 Assessment Tool 2
REVERSE ENGINEERING TASK

Name: Kaviya V **Reg No:** 192421022

Question 2: A software module's execution traces and code snippets are given, but no test documentation is available. Reverse engineer the white box testing approach by constructing the control flow graph from the given logic. Identify all decision points, loops, and independent execution paths. Evaluate the existing test adequacy in terms of coverage criteria. Reconstruct a complete path testing strategy ensuring full coverage. Present your findings with clear reasoning and structured representation.

1. Understanding the Problem:

The software module has code snippets and execution traces, but its test documentation is missing. Reverse engineering means examining the available code and traces to rebuild the testing model: identify program components, decisions, loops, and execution paths; compare the paths with the tests that were run; and design additional tests for uncovered logic. This is important because a module may appear to work for normal inputs while still containing defects in error handling, boundary conditions, or alternate branches.

The supplied screenshot shows the assessment rubric, not the actual code snippets or execution traces. Therefore, the example below demonstrates the required method using a simple input-processing module. For an exact answer, the example must be replaced with the real module logic.

2. System Decomposition and Example Logic:

Assume the module reads a list of values, validates the input, loops through the list, and classifies each value as positive or zero/negative. The main components are: input validation, loop control, value decision, processing statements, and final reporting.

START\nRead input list\nIF input is invalid: display error and END\nELSE set i = 0\nWHILE i < list length:\n IF value[i] > 0: process positive value\n ELSE: process zero/negative value\n i = i + 1\nDisplay success\nEND

3. Control Flow Graph (CFG):

Represent each statement or decision as a node and each possible transfer of control as an arrow.

Node	Statement or decision
1	Start and read input
2	Decision A: Is input valid?
3	Display error and end
4	Initialize i = 0
5	Decision B: Is i less than list length?
6	Decision C: Is the current value positive?
7	Process positive value
8	Process zero/negative value
9	Increment i
10	Display success and end

CFG edges: 1→2; 2 (invalid)→3; 2 (valid)→4; 4→5; 5 (true)→6; 5 (false)→10; 6 (true)→7; 6 (false)→8; 7→9; 8→9; 9→5. Nodes 3 and 10 are exit points.

Decision points are A (input validation), B (loop condition), and C (value classification). The loop starts at node 5 and repeats through node 9. The program can exit early for invalid input or finish normally when the loop condition becomes false.

4. Independent Paths and Cyclomatic Complexity:

An independent path includes at least one edge not covered by the previously selected paths. This example has three decision nodes, so its cyclomatic complexity is V(G) = D + 1 = 3 + 1 = 4. Four independent basis paths should therefore be tested.

P1 – Invalid input: 1→2 (invalid)→3→END.

P2 – Valid empty list: 1→2 (valid)→4→5 (false)→10→END.

P3 – One positive value: 1→2 (valid)→4→5 (true)→6 (true)→7→9→5 (false)→10→END.

P4 – One zero/negative value: 1→2 (valid)→4→5 (true)→6 (false)→8→9→5 (false)→10→END.

Because a loop may execute many times, it is not always practical to enumerate every complete path. Basis-path testing should be supplemented with loop tests for zero iterations, one iteration, multiple iterations, and input-size boundaries.

5. Evaluate Existing Test Adequacy:

Coverage criterion	Purpose	Potential coverage gap
Statement coverage	Runs every executable statement at least once.	Error-handling or one processing statement may be skipped.
Branch coverage	Executes both outcomes of every decision.	Invalid input, loop exit, or the alternate value branch may be missing.
Condition coverage	Makes each Boolean condition true and false.	A condition may only be tested with one result.
Path coverage	Exercises independent and important execution paths.	Only the normal successful path may have been tested.
Loop coverage	Tests zero, one, and multiple iterations.	Empty input and repeated processing may not be checked.

The exact coverage achieved cannot be calculated without the real traces and test results. To assess it, map every recorded trace to CFG nodes and edges, then mark covered and uncovered statements, branches, and paths. Statement coverage alone is insufficient: all statements might execute while one branch outcome remains untested. Full path coverage may be impossible for unbounded loops, so use basis paths and risk-based boundary tests.

6. Complete Path Testing Strategy:

1. Construct the CFG. Convert statements into nodes and control transfers into directed edges.
2. Identify decisions and loops. Record true/false outcomes and each loop's entry, body, increment, and exit.
3. Calculate complexity. Use V(G) = decision nodes + 1 for a simple connected graph, or E − N + 2P.
4. Create basis-path tests. Design at least one test for every independent path.
5. Add boundary and negative tests. Test invalid input, empty input, one value, multiple values, positive values, zero, negative values, and maximum permitted size.
6. Measure coverage. Use a coverage tool and a traceability matrix; investigate uncovered nodes and branches.
7. Retest and regress. After fixing defects, repeat the affected tests and run regression tests to check existing behavior.

7. Justification and Conclusion:

This strategy is effective because it reconstructs the internal structure instead of relying only on outputs. The CFG makes decisions and loops visible, cyclomatic complexity estimates the number of independent basis paths, and coverage measures reveal untested logic. Combining statement, branch, condition, path, and loop testing provides stronger evidence that normal and exceptional behavior have been checked. Retesting and regression testing also confirm that corrections work without introducing new defects.

In conclusion, the final reverse-engineering report should contain the CFG, decision and loop list, independent paths, a coverage-gap analysis, and a repeatable test plan. The example illustrates the method, but the final graph and paths must be adapted to the actual code snippets and execution traces from the question.